visCOS

Daniel Klotz, Johannes Wesemann, Simon Frey, Mathew Herrnegger

Contents

1 The Guide **1**
1.1 Introduction . . . 2
1.2 visCOS and other R-package . . . 2

2 cosdata **2**
2.1 Examples plots from cosdata . . . 4

3 Runoff Peaks **6**
3.1 Examples: . . . 6
3.2 Code . . . 9
3.3 References . . . 12

4 Objective Functions **12**
4.1 Code . . . 16

5 Exploring Objective Functions **22**
5.1 Example . . . 22
5.2 Code . . . 23

6 Comparisons: Exploring Objective Functions II **28**
6.1 Code . . . 28

1 The Guide

This is the guide and definition of visCOS.

visCOS stands for "visual comparison of observations and simulations". The package is still changing quite often. We therefore recommend to not use it yet.

1.1 Introduction

This guide defines the code for the R-package visCOS and provides instructions, examples and tips for using it.

The goal of visCOS is to provide a possibility to streamline the R based visual-analysis for the hydrological model COSERO. COSERO is a HBV-like distributed rainfall-runoff model, developed at the institute for water management, hydrology and hydraulic engineering at BOKU, Vienna. The name is an abbreviation for "Conceptual Semi-Distributed Rainfall Runoff Model".

visCOS is embedded into a broader effort that aims to establish practical and easy-to-use workflows between R and COSERO. The overall concept of that project is shown in **Figure xyz**. The R-package `coscos` provides the backbone of the projects. It provides the basic interface between COSERO and R, generic helper functions and primitives, and defines the options for all subsequent package. Currently, visCOS is the only package that builds on it, but hopefully
soon we will have more packages, ideas and scripts that are able to use it. s

Figure xyz

1.2 visCOS and other R-package

There exist many R-packages dedicated to hydrology.

Among then, several have provide help to the calibration of (conceptual) rainfall-runoff models. As their name suggests, these models relate the a rainfall input to a generated runoff for some given basins They have found a wide array of applications in hydrology. Form the analysis of catchment properties to climate impact studies, so to speak. From the point o view of the R package ecosystem there are to packages, which are of particular interest with regard to visCOS: The `hydroGOF` package and the `hydroTSM` package. The former provides the most commonly used (and discussed) objective functions. The latter, helps with the analysis, interpolation and the plotting of hydrological time series.

As an R-package that aim of visCOSis to provide visual aids for the parameter estimation process of multiple (related) basins or catchments.

Thus, topic wise, visCOS can be positioned somewhere in-between the two previously mentioned packages. visCOS is usable for conceptual rainfall runoff models in general, but almost all of its functionality has been derived from different applications of the COSERO model.

2 cosdata

For visCOS raw-data is data directly taken from COSERO. In general, raw-data can have many forms. The focus of visCOS are time series of observations, o, and model simulations, s. It assumes that the data is already loaded in to R.

R has many options to read data. Most file-formats can be accessed with the native `read.` functions (for an example enter `?read.table` in your R-terminal). However, for larger files we recommend the read functionality of the `data.table` package. In our tests it was fastest and most flexible choice. Lastly, we recommend `readr` package for well structured and large data.

After the data is read into R it needs to be transformed (*cooked*) into a rigidly defined format, which we call **cosdata**. The format has some redundant columns, but provides a large optionality. It is also a good middle ground between the output of COSERO and the formats used within the R ecosystem.

All cosdata formated data can be used for further exploratory analysis with visCOS.

NOW WE NEED TO EXPLAIN COSDATA!

Snipptes:

Currently visCOSonly allows to compare between *numbered catchments*! The data **must** include an integer number at the end of its name, e.g. `QObs_001` and `QSim_001`).

Additionally, `cos_data` data.frame needs to have two different definitions for the date of a given row (see: Introduction). One is based on an old way to write dates out in fortran code. In that case each column represents a time-resolution. In concrete, the following columns are used:

- *yyyy* - year,
- *mm* - month,
- *hh* - hour,
- *min* - minute.

The other format is a more modern way to define time information. That is, the `POSIXct` format (see: link). This format is a standard for `R` and has many usages (e.g. transforming your data frame into a time series).

visCOS can differentiate seasonal information on a monthly resolution. The in- and out-of-period markings are stored in a separate column (defined by `viscos_options("name_COSperiod")`). Here, each season is defined by an number (integer), which starts 1 and is raised for each new season. A 0 indicates the out-of-period rows.

If needed, the column-names of the cosdata can adapted by using the `viscos_options` function from `coscos` (alternatively one might prefer to set the column-name the `data.frame` accordingly). Calling the function `viscos_options()` will return a list with the available options. The available options are as following:

```r
# showing the options:
library(coscos)
library(magrittr)
viscos_options() %>% unlist(.)
```

```
##         name_o         name_s        name_lb        name_ub   name_COSyear
##         "qobs"         "qsim"           "lb"           "ub"         "yyyy"
##  name_COSmonth    name_COSday   name_COShour    name_COSmin  name_COSposix
##           "mm"           "dd"           "hh"          "min"    "posixdate"
## name_COSperiod      data_unit   missing_data        color_o        color_s
##       "period"      "(m^3/s)"         "-999"   "dodgerblue"       "orange"
##     of_limits1     of_limits2
##            "0"            "1"
```

An option can be changed by providing the new option-value as functional argument of the option. Here is an example for changing the *o* column color:

```r
viscos_options(color_o = "dodgerblue")
viscos_options( ) %>% unlist(.)
```

```
##         name_o         name_s        name_lb        name_ub   name_COSyear
##         "qobs"         "qsim"           "lb"           "ub"         "yyyy"
##  name_COSmonth    name_COSday   name_COShour    name_COSmin  name_COSposix
##           "mm"           "dd"           "hh"          "min"    "posixdate"
## name_COSperiod      data_unit   missing_data        color_o        color_s
##       "period"      "(m^3/s)"         "-999"   "dodgerblue"       "orange"
```

```
##     of_limits1     of_limits2
##            "0"            "1"
```

The `coscos` package also provides a function to get some exemplary data to use in visCOS. Throughout this guide we will use it to demonstrate the different functionalities of visCOS. The data can be loaded with the function `viscos_example`, e.g.:

```
options(width=80)
require(coscos)
#
raw_data_example <- viscos_example( )
head(raw_data_example)
```

```
##   yyyy mm dd hh min QOBS_0001 QOSI_0001 QSIM_0001 QOBS_0002 QOSI_0002 QSIM_0002
## 1 2007  1  1  0   0      2.98      3.48      3.48      2.56      3.11      3.11
## 2 2007  1  1  1   0      2.89      3.48      3.48      2.56      3.11      3.11
## 3 2007  1  1  2   0      2.64      3.48      3.48      2.57      3.11      3.11
## 4 2007  1  1  3   0      2.51      3.48      3.48      2.57      3.11      3.11
## 5 2007  1  1  4   0      2.42      3.48      3.48      2.57      3.11      3.11
## 6 2007  1  1  5   0      2.34      3.49      3.49      2.57      3.11      3.11
```

One can see that the example data still has some columns that are not needed for further analysis, i.e. the columns `QOSI_0001` and `QOSI_0002`. Furthermore, the `posixdate` and the `period` columns are still missing in the data. Again, the `coscos` provides a streamlined function, called `cook_cosdata` to get rid of the non-needed columns, complete the date columns and add the `period` column with pre-specified settings. With the raw-data example from before, the finished cosdata would look as following:

```
raw_data_example %>%
  cook_cosdata(.) %>%
  head(.)
```

```
## # A tibble: 6 × 11
##    yyyy    mm    dd    hh   min QOBS_0001 QSIM_0001 QOBS_0002 QSIM_0002
##   <int> <int> <int> <int> <int>     <dbl>     <dbl>     <dbl>     <dbl>
## 1  2007     1     1     0     0      2.98      3.48      2.56      3.11
## 2  2007     1     1     1     0      2.89      3.48      2.56      3.11
## 3  2007     1     1     2     0      2.64      3.48      2.57      3.11
## 4  2007     1     1     3     0      2.51      3.48      2.57      3.11
## 5  2007     1     1     4     0      2.42      3.48      2.57      3.11
## 6  2007     1     1     5     0      2.34      3.49      2.57      3.11
## # ... with 2 more variables: posixdate <dttm>, period <int>
```

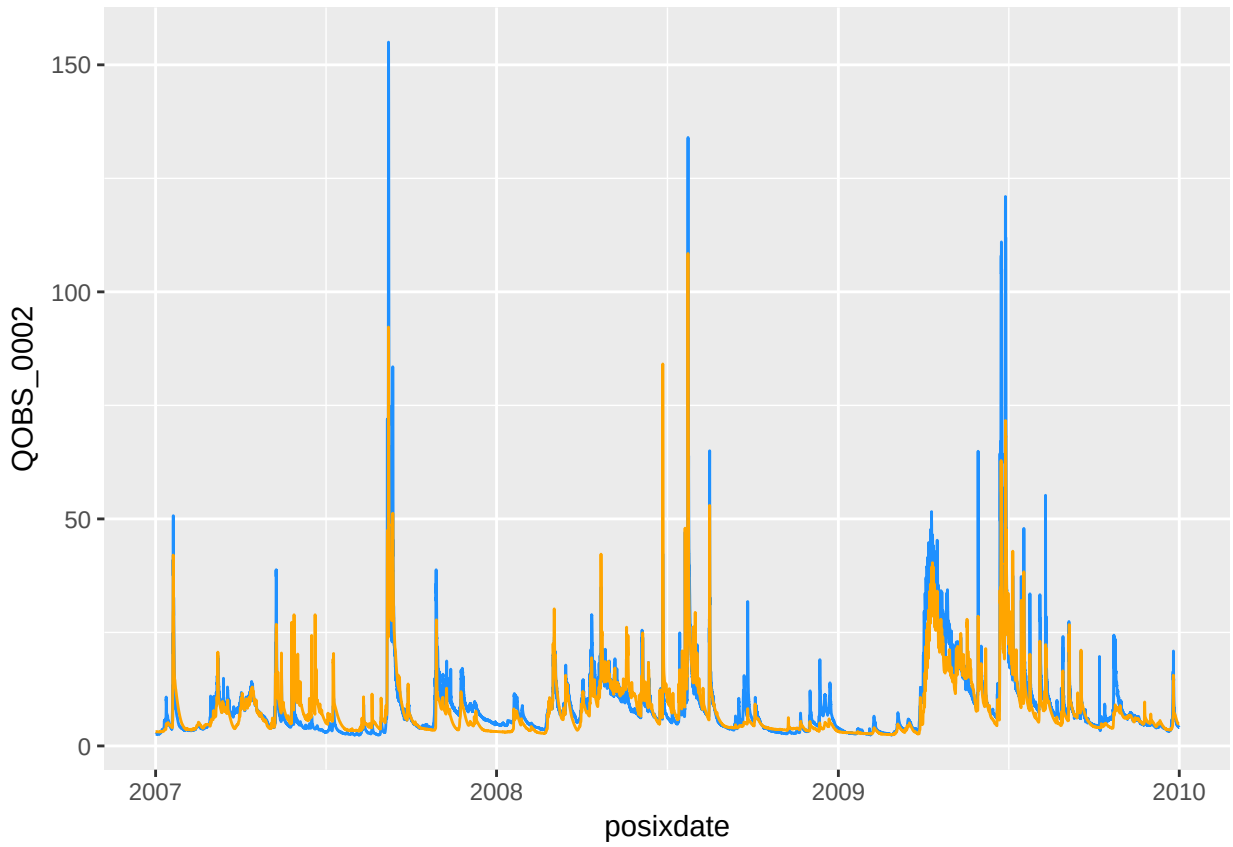
The `cook_cosdata` function can be very useful. If you are not sure that your data has the right format yet, just try to use `cook_cosdata` on it. The function is used at the start of almost all visCOS functions. Thus, it is very likely that your data is fine if `cook_cosdata` works on it.

2.1 Examples plots from cosdata

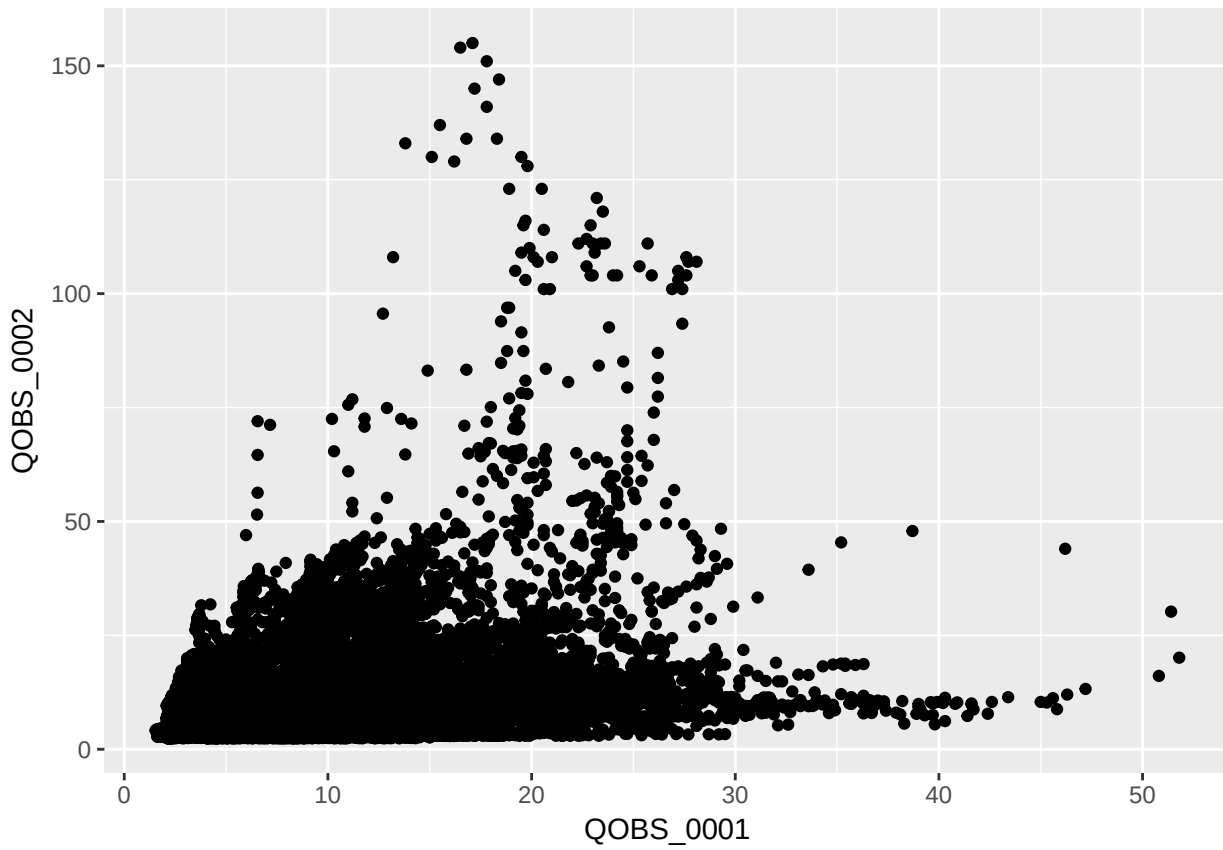
It is already possible to do a lot with just the cosdata. In the following are some examples:

```
library(ggplot2)
raw_data_example %>%
  cook_cosdata(.) %>%
  ggplot(.) +
    geom_line(aes(x = posixdate, y = QOBS_0002),
              color = viscos_options("color_o")) +
```

```
    geom_line(aes(x = posixdate, y = QSIM_0002),
              color = viscos_options("color_s"))
```




```
library(ggplot2)
raw_data_example %>%
  cook_cosdata(.) %>%
  ggplot(.) +
    geom_point(aes(x = QOBS_0001, y = QOBS_0002))
```



3 Runoff Peaks

The function `peack` lets users explore the highest events in among the available basins. It provides a list of ggplot2 plots, containing an overview plot (`overview`), a scatter plot (`scatter`) and detail plots of the individual events (`event_plot`). Instead of explaining the properties of each plot in detail it is best to get an intuition of the function by looking at some examples.

3.1 Examples:

For the examples 10 events are extracted from a runoff example

```
library(coscos)
library(visCOS)
cos_data <- viscos_example()
peakplots <- peack(cos_data, action = "plots", n_events = 10L)
```

The `peakplots` list does now contain plots for each basin within the `cos_data` data.frame:

```
names(peakplots)
```

```
## [1] "basin0001" "basin0002"
```

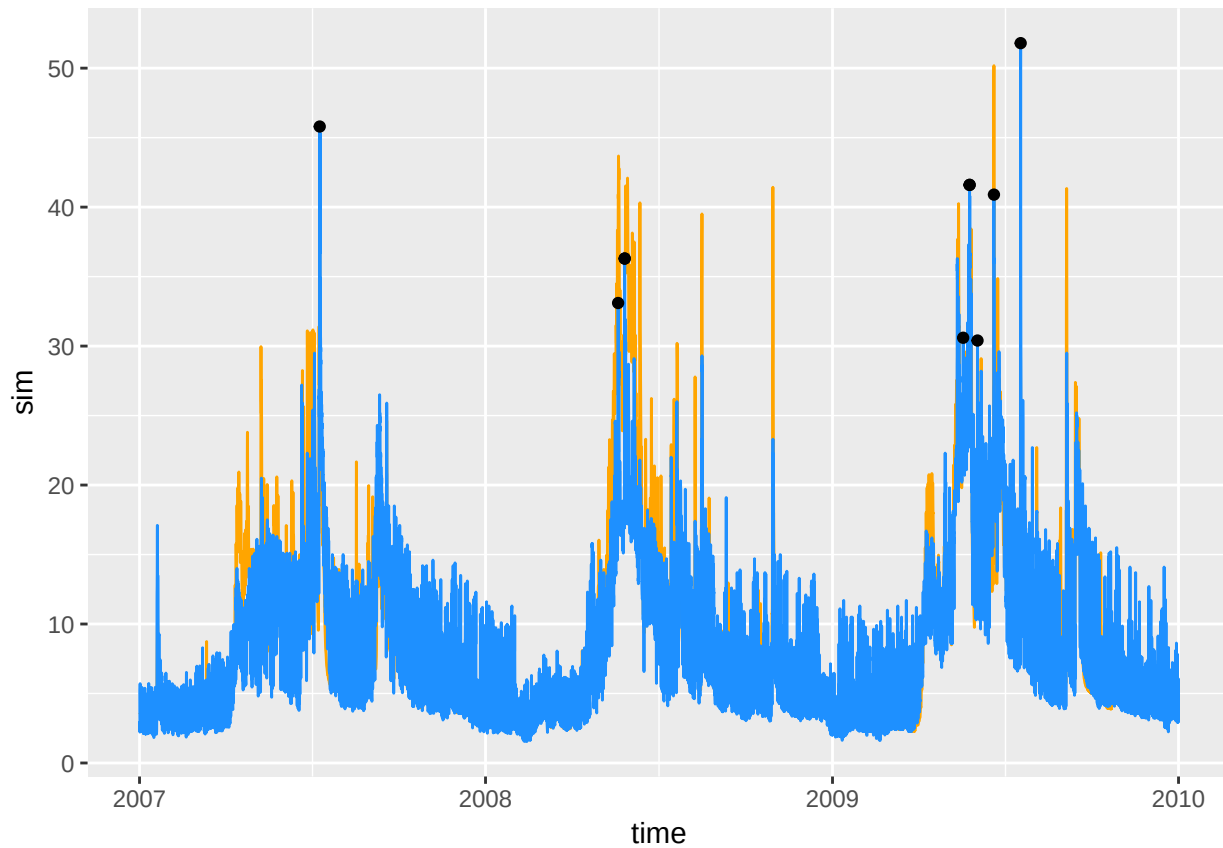
For each basin the a set of plots (`overview`,`scatter`,`event_plot`) are saved within a list for each basin. In the following the plots for basin 1 are shown:

```r
names(peakplots$basin0001)
```

```
##  [1] "overview"     "scatter"      "event_plot1"  "event_plot2"  "event_plot3"
##  [6] "event_plot4"  "event_plot5"  "event_plot6"  "event_plot7"  "event_plot8"
## [11] "event_plot9"  "event_plot10"
```

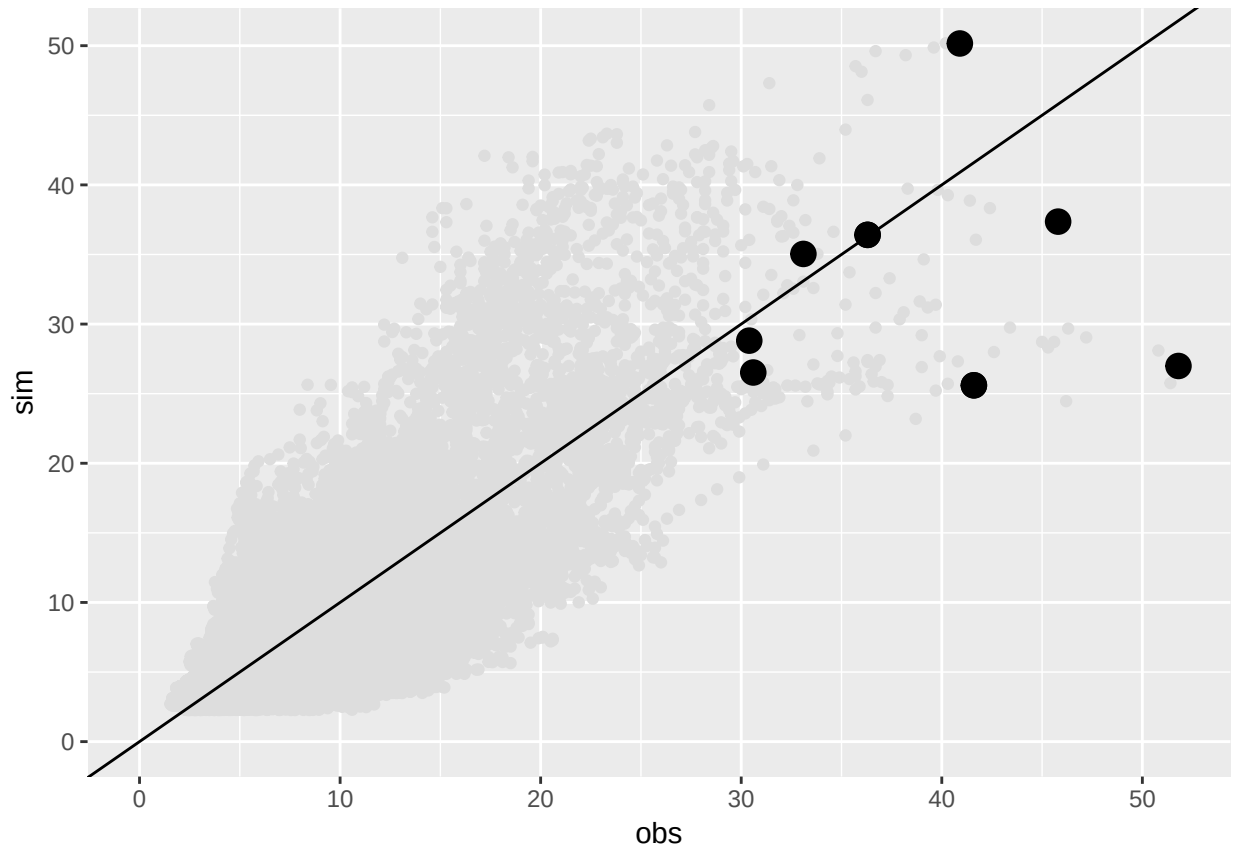
The `overview` plot shows the entire time series of `data1` and `data2` of the basin. The found events are marked with black dots. The `overview` plot for basin 1 is:

```r
peakplots$basin0001$overview
```



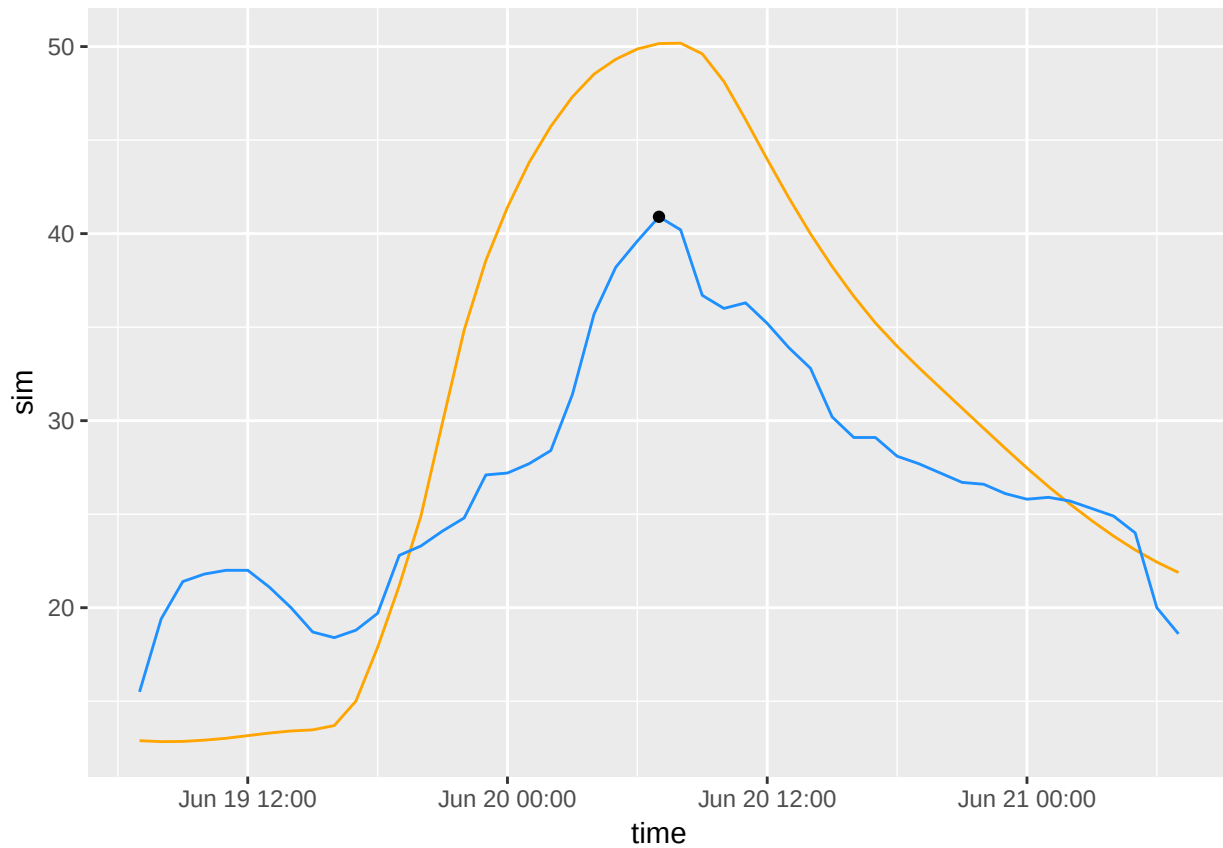
The `scatter` plot shows the found events within a scatter plot, where `data1` is the x-axis and `data2` on the y-axis. In the following an example for basin 1 is given.

```r
peakplots$basin0001$scatter
```



Detail plots for each of the found events are given in form of the `event_plot` objects. Here an example:

```
peakplots$basin0001$event_plot5
```



3.2 Code

This part of the document defines the code of `peack`

```
# ----------------------------------------------------------------------------
#' Event-wise Exploration of the high-runoffs
#'
#' @export
#'
#' @import ggplot2
#' @import dplyr
#' @import magrittr
#' @importFrom tibble tibble
peack <- function(cos_data,
                  action = "compute",
                  n_events= 10L,
                  window_size = 24L,
                  opts = coscos::viscos_options()) {
  # pre:
  action_string <- match.call(expand.dots = FALSE) %>%
    .["action"] %>%
    .[[1]] %>%
    as.character(.)
  if (length(action_string) == 0) {
    action_string = "compute"
  }
```

```
  #
  cosdata <- coscos::cook_cosdata(cos_data)
  n_events_int <- as.integer(n_events)
  window_size_int <- as.integer(window_size)
  if (is.na(n_events_int)  |
      is.nan(n_events_int) |
      is.infinite(n_events_int) |
      !is.integer(n_events_int) ) {
    stop("n_events is ill defined")
  }
  if (is.na(window_size)  |
      is.nan(window_size) |
      is.infinite(window_size) |
      !is.integer(window_size) ) {
    stop("window_size is ill defined")
  }
  data1 <- cosdata %>%
    dplyr::select( starts_with(opts[["name_o"]]) )
  data2 <- cosdata %>%
    dplyr::select( starts_with(opts[["name_s"]]) )
  data_posix <- cosdata %>%
    dplyr::select(matches(opts[["name_COSposix"]]))
  data_numbers <- names(data1) %>%
    gsub(opts[["name_o"]], "", ., ignore.case = TRUE) %>%
    gsub("\\D","",.,ignore.case = TRUE)
  result_list <- lapply(1:ncol(data1),
                 function(x) peack_one_basin(data1[ ,x],
                                             data2[ ,x],
                                             data_posix,
                                             n_events_int,
                                             window_size_int,
                                             action_string,
                                             opts)) %>%
    set_names(., paste("basin", data_numbers, sep = ""))
  return(result_list)
}
```

3.2.1 Peak-Plots for a signle Basin

This is the function for generating the different plots for one basin. At first the provided time series are grouped into a `tibble`, then the peaks of the observations are obtained via the peak_finder function and organised. Then `ggplot2` is used for plotting.

```
peack_one_basin <- function(qobs,
                            qsim,
                            data_posix,
                            n_events_int,
                            window_size_int,
                            action_string,
                            opts) {
  single_data <- cbind(data_posix,qobs,qsim) %>%
    magrittr::set_names(c("time","obs","sim"))
  #
```

```r
  peak_idx <- find_peaks(single_data$obs,m = window_size_int)
  peak_organised <- tibble::tibble(idx = as.integer(peak_idx),
                                   time = single_data$time[peak_idx],
                                   peak_obs = single_data$obs[peak_idx],
                                   peak_sim = single_data$sim[peak_idx])
  highest_peaks_organised <- peak_organised$peak_obs %>%
    sort(decreasing = TRUE) %>%
    .[1:n_events_int] %>%
    match(.,peak_organised$peak_obs) %>%
    peak_organised[., ]
  #
  if (action_string == "compute") {
    return(highest_peaks_organised)
  } else if (action_string == "plots") {
  overview_plot <- ggplot( ) +
    geom_line(data = single_data, aes(x = time, y = sim),
              col = opts[["color_s"]]) +
    geom_line(data = single_data, aes(x = time, y = obs),
              col = opts[["color_o"]]) +
    geom_point(data = highest_peaks_organised, aes(time, peak_obs))
  overview_scatter <- ggplot() +
    geom_point(data = single_data, aes(obs,sim), color = "#DDDDDD") +
    geom_abline() +
    geom_point(data = highest_peaks_organised, aes(peak_obs,peak_sim), size = 4) +
    expand_limits(x = 0, y = 0)
  sub_plots <- lapply(
    1:nrow(highest_peaks_organised),
    function(x) sub_peakplot_fun(x,
                                 window_size_int,
                                 highest_peaks_organised[x,]$idx,
                                 single_data,
                                 opts)
    ) %>%
    set_names(.,paste("event_plot", 1:length(.), sep = ""))
  return(overview = append(list(overview = overview_plot,
                                scatter = overview_scatter), sub_plots))
  } else {
    stop("The chosen 'action' does not exist.")
  }

}
```

3.2.2 Peak Finding Function

The function for finding the peaks was proposed and developed by the cross validated user "stas g" in this thread.
This is by far not the only option/possibility to approach the peak finding task. Other nice ideas for finding peaks can be found in this cross validated thread.

```r
####
# peak finder function:
find_peaks <- function(x, m = 3){
  shape <- diff(sign(diff(x, na.pad = FALSE)))
```

```
  pks <- sapply(which(shape < 0), FUN = function(i){
    z <- i - m + 1
    z <- ifelse(z > 0, z, 1)
    w <- i + m + 1
    w <- ifelse(w < length(x), w, length(x))
    if(all(x[c(z : i, (i + 2) : w)] <= x[i + 1])) return(i + 1) else return(numeric(0))
  })
  pks <- unlist(pks)
  pks
}
```

3.2.3 Subplot Function

This function is a wrapper around `ggplot`, which is used to generate the individual event plots.

```
####
# sub plot function:
sub_peakplot_fun <- function(x, window_size, idx, peak_data, opts) {
  point <- peak_data[idx, ]
  plot_sub <- peak_data[(idx - window_size):(idx + window_size), ] %>%
    ggplot(data = .) +
    geom_line(aes(x = time, y = sim),
              col = opts[["color_s"]]) +
    geom_line(aes(x = time, y = obs),
              col = opts[["color_o"]]) +
    geom_point(data = point, aes(time, obs))
  return(plot_sub)
}
```

3.3 References

- http://stats.stackexchange.com/questions/22974/how-to-find-local-peaks-valleys-in-a-series-of-data (checked 12/2016)
- http://stats.stackexchange.com/questions/36309/how-do-i-find-peaks-in-a-dataset (checked 12/2016)

4 Objective Functions

This chapter presents the objective function summary functions in visCOS. They compute basin-wise objective functions of the overall data and for the individual basins.

As explained in previous section objective functions are a pivotal part of model calibration. If no specific options are chosen visCOS uses 4 objective functions, i.e.: NSE, KGE, Pearson's Correlation and the Percentage Bias (the function definitions have been given in the chapter about distance metrics).

The objective functions for the overall data and the marked periods can be computed through the function `judge_compute`. Additionally, visCOS already provides two different options to create plots for the main objective functions: `judge_rasterplot` and `judge_barplot`. Both functions create a list with `ggplot` figures. Each entry in the list corresponds to one of the main objective functions and both lists can be saved to html embedded pictures with the `serve` function.

The `judge` function provides a wrapper for all other `judge_` and `of_` functions. The selection is done via the non-standardly-evaluated `type` argument (search for lazy-evaluation if you want to know more. This implies, among other things, that

In the following are some examples:

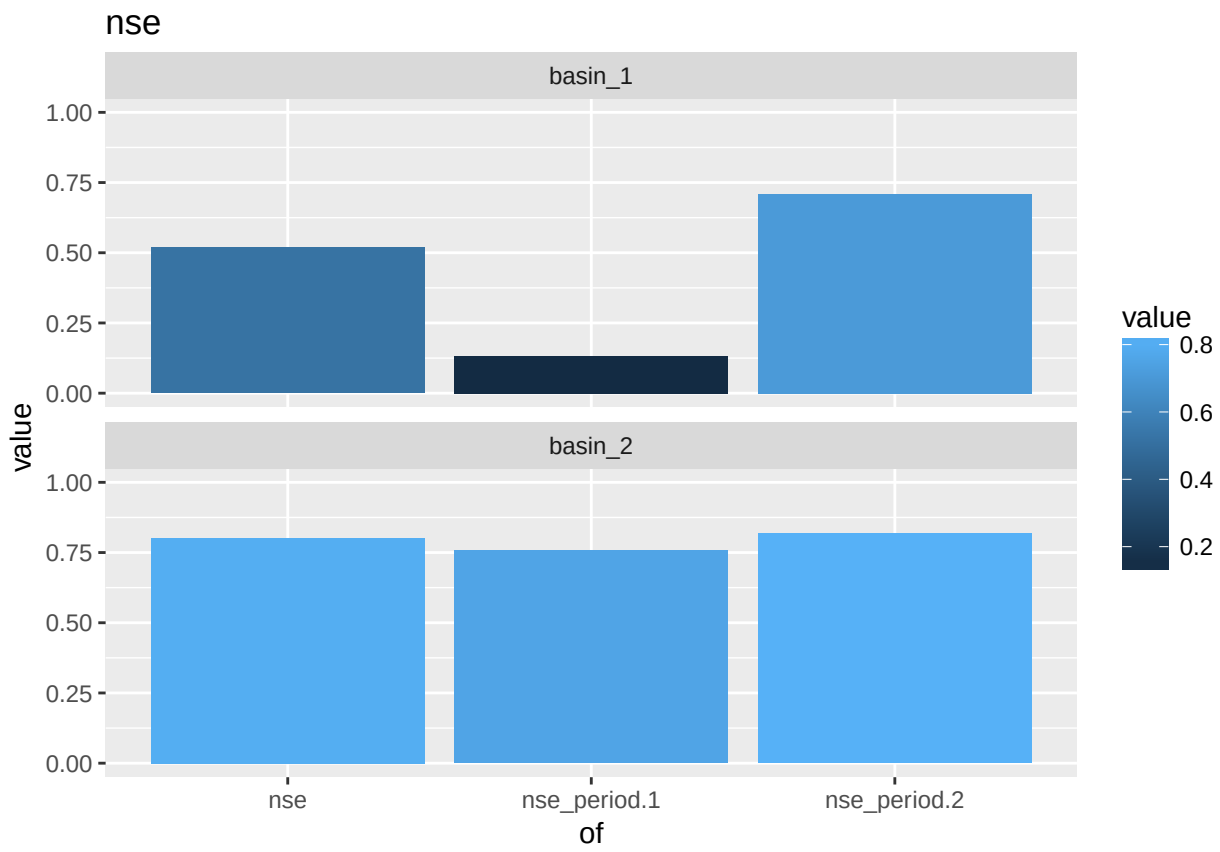
Example 1. Computing the main objective functions

```
library(magrittr)
library(coscos)
library(visCOS)
#
viscos_example( ) %>%
  judge(.) %>%
  print(.)
```

```
## # A tibble: 12 × 3
##                of    basin_1     basin_2
## *           <chr>      <dbl>       <dbl>
## 1             nse  0.5177309   0.8012919
## 2             kge  0.7246877   0.7489862
## 3           pbias  0.2506191 -12.5298276
## 4            corr  0.8177663   0.9107391
## 5   corr_period.1  0.8243198   0.8806770
## 6   corr_period.2  0.8577983   0.9385221
## 7    kge_period.1  0.4138710   0.8487406
## 8    kge_period.2  0.8419666   0.6820128
## 9    nse_period.1  0.1326604   0.7566200
## 10   nse_period.2  0.7096110   0.8168895
## 11 pbias_period.1  8.8238223  -8.6127526
## 12 pbias_period.2 -6.8569950 -15.9030884
```

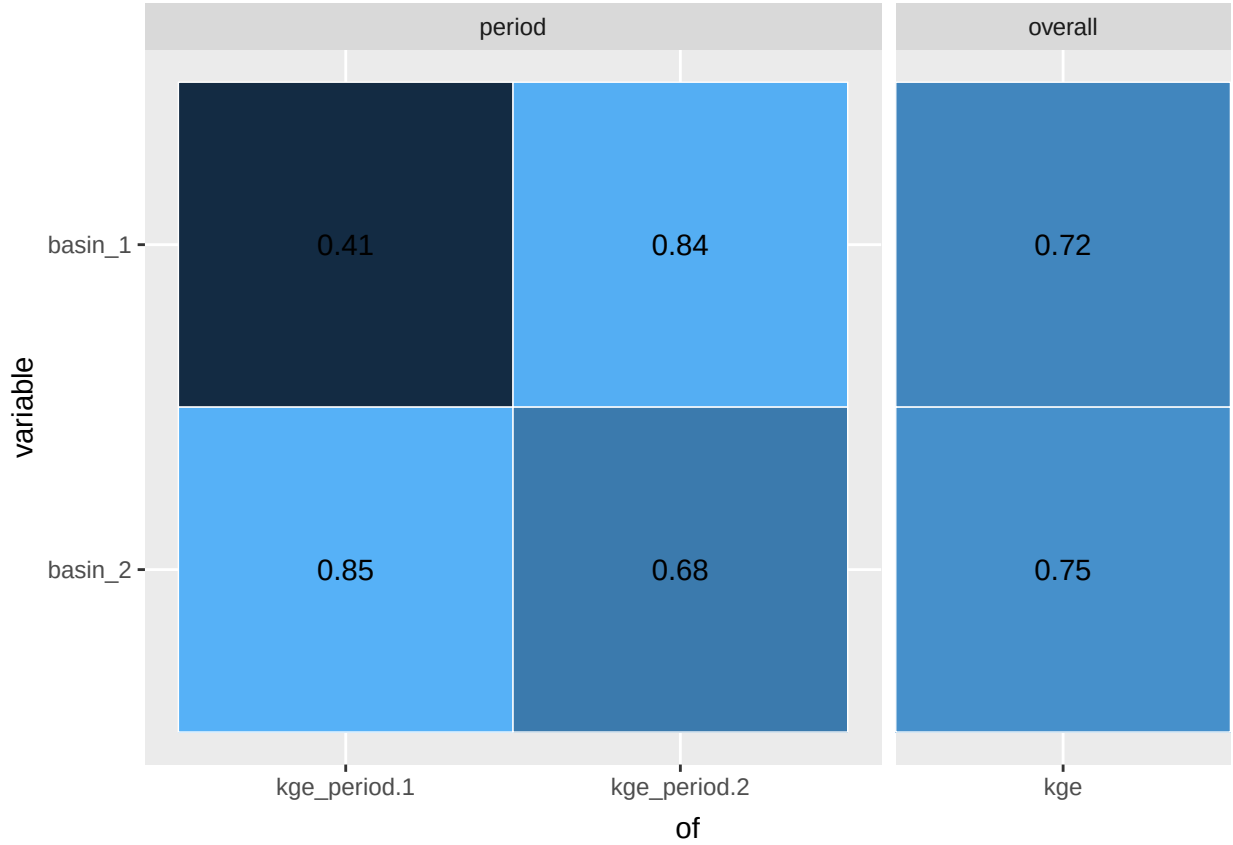
Example 2. Plotting the results of main objective function with bar plots:

```
viscos_example() %>%
  judge(.,barplot) %>%
  .[[1]] %>%
  plot(.)
```



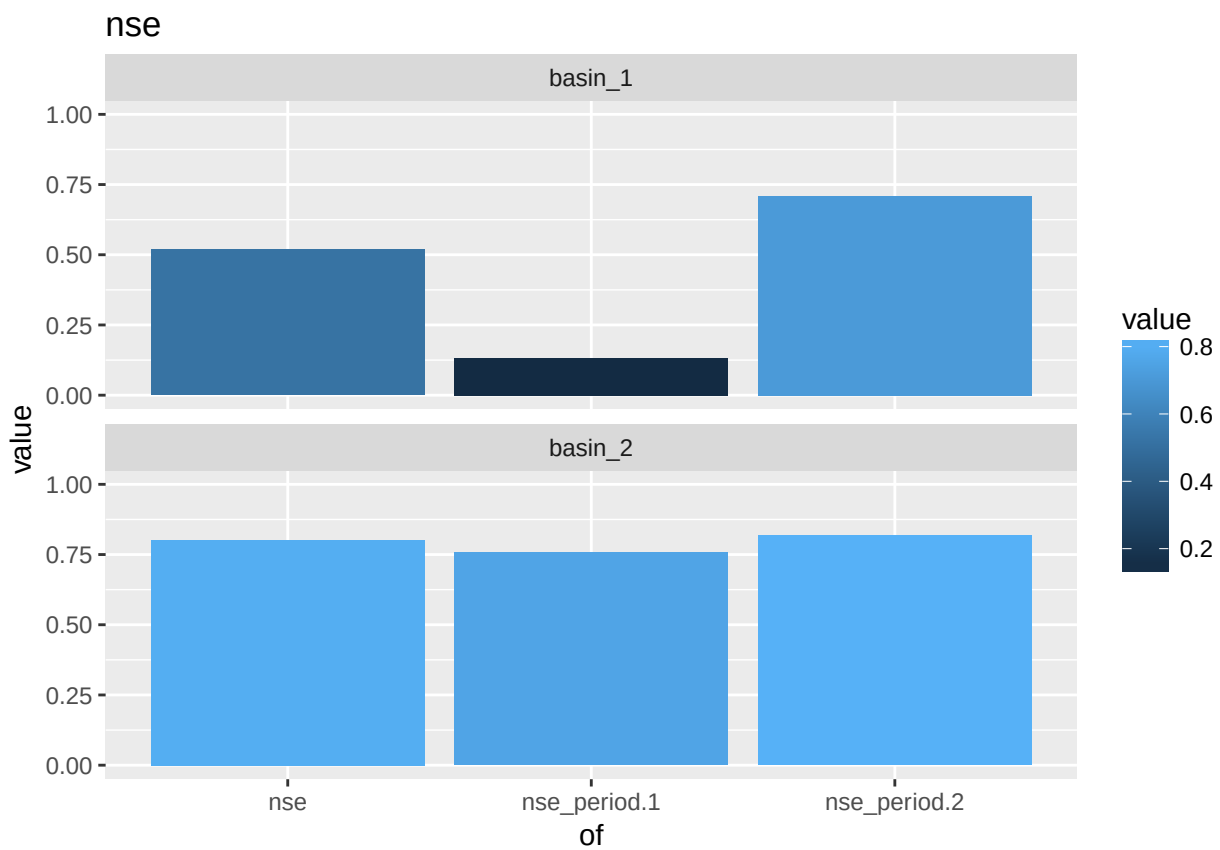
Example 3. Plotting the results of main objective function with a raster:

```
viscos_example() %>%
  judge(.,rasterplot) %>%
  .[[2]] %>%
  plot(.)
```



Example 4. Using the `judge_` functionality:

```
viscos_example() %>%
  judge_barplot(.) %>%
  .[[1]] %>%
  plot(.)
```



4.1 Code

```
# ---------------------------------------------------------------------------
# Code for the Main Objective Functions (main_of)
# authors: Daniel Klotz, Johannes Wesemann, Mathew Herrnegger
# !!!!
```

```
                  type = "compute",
                  of_metrics = list(nse   = coscos::of_nse,
                                    kge   = coscos::of_kge,
                                    pbias = coscos::of_pbias,
                                    corr  = coscos::of_cor),
                  opts = coscos::viscos_options(),
                  ...) {
  # lazy evaluation: -------------------------------------------------------
  le_dots <-  match.call(expand.dots = FALSE) %>%
    .["..."] %>%
    .[[1]]
  le_type <- match.call(expand.dots = FALSE) %>%
    .["type"] %>%
    .[[1]] %>%
    as.character(.)
  if (length(le_type) == 0) {
    le_type = "compute"
  }
  if (class(le_type) != "character") {
    stop("Cannot resolve 'type' argument. Try to provide a character.")
  }
  # compute options: -------------------------------------------------------
  if (le_type == "compute") {
    judge_compute(cosdata, of_metrics)
  } else if (le_type == "judge_barplot" | le_type == "barplot") {
    judge_barplot(cosdata, of_metrics)
  } else if (le_type == "judge_rasterplot" | le_type == "rasterplot") {
    judge_rasterplot(cosdata, of_metrics)
  } else if (le_type == "of_explore" | le_type == "explore") {
    judge_explore(cosdata, of_metrics)
  } else if (le_type == "of_compare" | le_type == "compare") {
    cosdata2 <- NULL
    if ("cos_data2" %in% names(le_dots)) {
      cos_data2 <- eval(le_dots$cos_data2)
    }
    judge_compare(d1 = cosdata,
                  d2 = cosdata2)
  } else {
    stop("There is no option (`type`) called" %&&% le_type)
  }
}
```

4.1.2 Computation

The purpose of this function is to extract a the main of objective functions from the `COSERO` data.frame. The objective functions are extracted for each basin separately and computed for the entire length of the data, as well as for each period separately.

The computational part of the function works as follows. In step (I) the non-marked periods of `cos_data` (columns of `viscos_options("name_COSperiod")` which are smaller then 0) are excluded from further calculations. The thereby obtained data.frame is named `evaluation_data`.

```r
# ----------------------------------------------------------------------------
#' Get basic objective function for cos_data
#'
#' Calculate basic objective functions(NSE, KGE, percentage BIAS, Correlation)
#' for every basin and the chosen periods.
#'
#' @param cos_data cos_data data.frame.
#' @return list of basic objective function evaluated for the different
#' hydrological years and over the whole timespan.
#'
#' @import pasta
#' @import tibble
#' @importFrom purrr map_df
#' @importFrom dplyr select filter starts_with
#' @importFrom magrittr set_names
judge_compute <- function(cosdata,
                          of_metrics = list(nse   = coscos::of_nse,
                                            kge   = coscos::of_kge,
                                            pbias = coscos::of_pbias,
                                            corr  = coscos::of_cor),
                       opts = coscos::viscos_options()) {
  # pre: ==================================================================
  cos_data <- coscos::cook_cosdata(cosdata)
  name_period <- opts[["name_COSperiod"]]
  name_o <- opts[["name_o"]]
  name_s <- opts[["name_s"]]
  #
  if( !(class(of_metrics) == "list") ) {
    of_metrics <- list(of_metrics)
  }
  if (is.null(names(of_metrics))){
    d_names <- "d" %&% 1:length(of_metrics)
  } else {
    d_names <- names(of_metrics)
  }
  #
  evaluation_data <- cos_data[cos_data[[name_period]] > 0, ]
  number_judge_basins <- evaluation_data %>%
    names(.) %>%
    unique(.) %>%
    grepl(name_o, ., ignore.case = TRUE) %>%
    sum(.)
  data_periods <- evaluation_data %>%
    .[[name_period]] %>%
    unique(.)
  number_judge_periods <- data_periods %>% length

  # compute main-of for entire data: ========================================
  d_mean <- purrr::map_df(of_metrics, function(judge_, x, y) as.numeric(judge_(x,y)),
                 x = dplyr::select(evaluation_data,dplyr::starts_with(name_o)) %>% unname(.),
                 y = dplyr::select(evaluation_data,dplyr::starts_with(name_s)) %>% unname(.) ) %>%
    t(.) %>%
    as_tibble(.) %>%
```

```r
    magrittr::set_names(., "basin" %_% 1:number_judge_basins) %>%
    cbind.data.frame(of = d_names,.)
  # compute periodwise main-of: ==========================================
  period_compute <- function(k) {
    o_pick <- dplyr::filter(evaluation_data,period == data_periods[k]) %>%
      dplyr::select(.,starts_with(name_o)) %>%
      unname(.)
    s_pick <- dplyr::filter(evaluation_data,period == data_periods[k]) %>%
      dplyr::select(.,starts_with(name_s)) %>%
      unname(.)
    d_measures <- purrr::map_df(of_metrics, function(judge_,x,y) as.numeric(judge_(x,y)),
                 x = o_pick,
                 y = s_pick ) %>% t()
    return(measure = d_measures)
  }
  d_names_periods <- d_names %_% "period" %.% rep(1:number_judge_periods, each = length(d_names))
  d_periods <- lapply(1:number_judge_periods, period_compute) %>%
    do.call(rbind,.) %>%
    as_tibble(.) %>%
    magrittr::set_names("basin" %_% 1:number_judge_basins) %>%
    cbind(of = d_names_periods,.) %>%
    .[order(.$of), ]
  #
  judge_all <- rbind(d_mean,d_periods) %>%
    as_tibble(.)
  judge_all$of <- as.character(judge_all$of)
  return(judge_all)
}
```

4.1.3 Bar Plots

```r
# ---------------------------------------------------------------------------
#' Bar plot for the Main Objective Function Values
#'
#' @rdname judge_overview
#'
#' @export
judge_barplot <- function(cosdata,
                          of_metrics = list(nse   = coscos::of_nse,
                                            kge   = coscos::of_kge,
                                            pbias = coscos::of_pbias,
                                            corr  = coscos::of_cor)) {
  # pre: ====================================================================
  cos_data <- coscos::cook_cosdata(cosdata)
  if(!(class(of_metrics) == "list")) {
    of_metrics <- list(of_metrics)
  }
  if (is.null(names(of_metrics))){
    d_names <- "d" %&% 1:length(of_metrics)
  } else {
    d_names <- names(of_metrics)
  }
```

```r
  # functions: ==============================================================
  assign_ofgroups <- function(judge_melted,judge_names) {
    judge_string <- as.character(judge_melted$of)
    judge_melted$judge_group <- judge_string %>%
      replace(., startsWith(judge_string,judge_names[1]), judge_names[1]) %>%
      replace(., startsWith(judge_string,judge_names[2]), judge_names[2]) %>%
      replace(., startsWith(judge_string,judge_names[3]), judge_names[3]) %>%
      replace(., startsWith(judge_string,judge_names[4]), judge_names[4])
    return(judge_melted)
  }
  # plot-list function:
  barplot_fun <- function(judge_name,judge_melted) {
    judge_to_plot <- judge_melted %>% filter( judge_group == judge_name)
    # bad solution, but will be fine for now?
    if (judge_name == "pbias") {
      gglimits <- c(-viscos_options("of_limits")[2]*100,
                    viscos_options("of_limits")[2]*100)
    } else {
      gglimits <- viscos_options("of_limits")
    }
    plt_out <- ggplot(data = judge_to_plot) +
      geom_bar(stat = "identity",
               position = "identity",
               aes(x = of, y = value, fill = value)) +
      facet_wrap(~ variable, ncol = 1) +
      ggtitle(judge_name) +
      ylim(gglimits)
    return(plt_out)
  }
  # computations: ===========================================================
  judge_data <- judge_compute(cos_data,of_metrics)
  num_basins <- ncol(judge_data) - 1
  judge_melted <- suppressMessages( reshape2::melt(judge_data) ) %>%
    assign_ofgroups(.,d_names)
  # plotting ================================================================
  plot_list <- lapply(d_names, function(x) barplot_fun(x,judge_melted)) %>%
    magrittr::set_names(d_names)
  return(plot_list)
}
```

4.1.4 Raster Plots

```r
#' Bar plot for the Main Objective Function Values
#'
#' @rdname judge_overview
#' @import pasta
#' @export
judge_rasterplot <- function(cos_data,
                             of_metrics = list(nse   = coscos::of_nse,
                                               kge   = coscos::of_kge,
                                               pbias = coscos::of_pbias,
                                               corr  = coscos::of_cor)) {
```

```r
  # def: ==================================================================
  build_tibble(cos_data)
  if(!(class(of_metrics) == "list")) {
    of_metrics <- list(of_metrics)
  }
  if (is.null(names(of_metrics))){
    d_names <- "d" %&% 1:length(of_metrics)
  } else {
    d_names <- names(of_metrics)
  }
  # computations: ========================================================
  regex_main_of <- d_names %.% "*"
  judge_data <- judge_compute(cos_data, of_metrics)
  #
  plot_list <- lapply(regex_main_of,function(x) plot_fun_raster(x,judge_data)) %>%
    magrittr::set_names(d_names)
  return(plot_list)
}

# plot function ---------------------------------------------------------
plot_fun_raster <- function(regex_single_of,judge_data) {
  # function definitions ===================================================
  extract_single_of <- function(judge_data){
    idx <- grep(regex_single_of,judge_data$of)
    return(judge_data[idx, ])
  }
  add_facet_info <- function(judge_data) {
    facet_column <- nrow(judge_data) %>%
      magrittr::subtract(1) %>%
      rep("period",.) %>%
      c("overall",.)
    return( cbind(judge_data,facets = facet_column) )
  }
  reverse_basin_levels <- function(prepared_data) {
    prepared_data$variable <- factor(prepared_data$variable,
                                     levels = prepared_data$variable %>%
                                       levels() %>%
                                       rev()
                                     )
    return(prepared_data)
  }
  reverse_facetting_levels <- function(prepared_data) {
    prepared_data$facets <- factor(prepared_data$facets,
                                   levels = prepared_data$facets %>%
                                     levels() %>%
                                     rev()
                                   )
    return(prepared_data)
  }
  bind_and_round_value <- function(of,gglimits,digits) {
    dplyr::mutate(of,
                  value = pmax(value,gglimits[1]) %>%
                    pmin(.,gglimits[2]) %>%
```

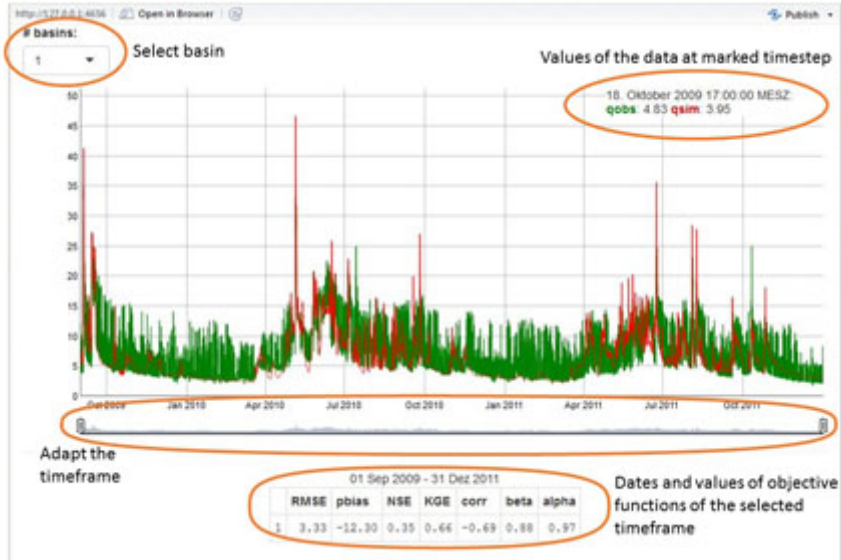
```
                    round(.,digits)
                  )
  }
  # computation ==============================================================
  if (regex_single_of == "p_bias.*") {
    # pbias has different limits :(
    gglimits <- c(-viscos_options("of_limits")[2]*100,
                  viscos_options("of_limits")[2]*100)
  } else {
    gglimits <- viscos_options("of_limits")
  }
  #
  prepared_data <- judge_data %>%
    extract_single_of() %>%
    add_facet_info() %>%
    reshape2::melt(., id.vars = c("of","facets")) %>%
    reverse_basin_levels() %>%
    reverse_facetting_levels() %>%
    bind_and_round_value(.,gglimits,2)
  # ggplot ====================================================================
  plt_out <- ggplot(prepared_data,
                    aes(of,variable, fill = value),
                    environmnet = environment()) +
    geom_raster(position = "identity") +
    coord_fixed(ratio = 5)  +
    facet_grid(~ facets,  scales = "free_x", space = "free") +
    theme( legend.position = "none")  +
    geom_tile(color = "white", size = 0.25 ) +
    geom_text(aes(of,variable, label = as.character(sprintf("%0.2f",value,2))),
              color = "black")
  return(plt_out)
}
```

5 Exploring Objective Functions

This section defines the code of a shiny gadget. It enables the interactive exploration of (hydro-) graphs for the different basins. The gadget shows always the corresponding objective function for the selected graph. Furthermore, one can get the selected data by clicking on "done" at the end of a session. The following examples provide a good overview of what the function can do.

5.1 Example

This chapter gives examples of `judge_explore`. For the pre-requirements take a look at the introduction. Running the `judge_explore` function without any options opens a shiny gadget in the viewer:



Information on the *objective functions* can be found here.

```
viscos_options(color_o = "green", color_s = "red")
judge(runoff_example,explore)
```

Users can select different basins via the selection box (*# basins:*) on the top-left and interactively zoom and move the graph in the center by clicking on it or moving the date switches below the graph. While doing so the objective functions (presented in the table below) are re-calculated for the chosen time window.

5.2 Code

In the following paragraphs the code of the shiny app is defined. The computations of the app are defined in the `server` part and the appearance in the `ui`.

```
# ------------------------------------------------------------------------
#' Explore with Objective Functions
#'
#' Runs a Shiny Gadget which can be used to get an overview of a cos_data time
#' series object.
#'
#' @import shiny
#' @import miniUI
#' @import xts
#' @import dplyr
#' @import magrittr
#' @import dygraphs
#' @import pasta
#' @importFrom purrr map_df
#'
#' @export
#'
#' @rdname judge
#'
#' @examples
#' # get example data,
#' # explore the model performance
```

```r
  #' cos_data <- get_viscos_example()
  #' judge_explore(cos_data)
judge_explore <- function(cos_data,
                          of_metrics = list(nse   = coscos::of_nse,
                                            kge   = coscos::of_kge,
                                            pbias = coscos::of_pbias,
                                            corr  = coscos::of_cor),
                          opts =coscos::viscos_options()
                          ) {
  # (I) pre-sets: ==============================================================
  name_o <- opts[["name_o"]]
  name_s <- opts[["name_s"]]
  name_lb <- opts[["name_lb"]]
  name_ub <-  opts[["name_ub"]]
  #
  if (is.null(names(of_metrics))){
    names(of_metrics) <- paste("of", 1:length(of_metrics), sep = "_")
  }
  clean_cos_data <- cos_data %>%
    coscos::cook_cosdata(.) %>%
    coscos::remove_leading_zeros(.)
  #
  names_data <- names(clean_cos_data) %>% tolower(.)
  number_lb <- grepl(name_lb,
                     names_data,
                     ignore.case = TRUE) %>% sum(.)
  number_ub <- grepl(name_ub,
                     names_data,
                     ignore.case = TRUE) %>% sum(.)
  plot_bounds <- FALSE
  if( (number_lb > 0) & (number_ub > 0)) {
    number_obs <- grepl(name_o,
                        names_data,
                        ignore.case = TRUE) %>% sum(.)
    number_sim <- grepl(name_s,
                        names_data,
                        ignore.case = TRUE) %>% sum(.)
    if (number_lb != number_ub) {
      stop("number of available bounds is not the same!" %&&%
             "#lb=" %&% number_lb %&&%
             "#ub=" %&% number_ub)
    } else if ((number_lb != number_obs) | (number_lb != number_sim)) {
      stop("Number of bounds is not the same as the o/s data!" %&&%
            "#bounds=" %&% number_lb %&&%
             "#obs=" %&% number_obs %&&%
             "#sim=" %&% number_sim)
    } else {
      plot_bounds <- TRUE # switch: plot bounds
    }
  }
  # basic stuff: ===============================================================
  idx_names <- grepl(name_o,
                     names_data,
```

```
                    ignore.case = TRUE)
d_nums <- names_data %>%
    .[idx_names] %>%
    gsub("\\D","",.) %>%
    as.integer(.) %>%
    unique(.)
# (V) Define App: ========================================================
server <- function(input, output, session) {
  # (a) get needed strings: ##############################################
  unique_data_names <- gsub("\\d","",names_data) %>%
    unique(.)
  x_string <- unique_data_names[ grep(name_o,
                                      unique_data_names) ]
  y_string <- unique_data_names[ grep(name_s,
                                      unique_data_names) ]
  if (plot_bounds) {
    lb_string <-  unique_data_names[ grep(name_lb,
                                          unique_data_names) ]
    ub_string <-  unique_data_names[ grep(name_ub,
                                          unique_data_names) ]
  }
  # (b) select data:
  # note: the regular expressions "$" terminates the searchstring
  selector_x <- reactive({ x_string %&% input$basin_num %&% "$" })
  selector_y <- reactive({ y_string %&% input$basin_num %&% "$" })
  selector_lb <- reactive({
    if(plot_bounds){
      lb_string %&% input$basin_num %&% "$"
    } else {
      NA
    }
  })
  selector_ub <- reactive({
    if(plot_bounds){
      ub_string %&% input$basin_num %&% "$"
    } else {
      NA
    }
  })
  selected_data <- reactive({
    if (plot_bounds) {
      clean_cos_data %>%
        select(matches( selector_x() ),
               matches( selector_y() ),
               matches( selector_lb() ),
               matches( selector_ub() )
               ) %>%
        select(x = matches( selector_x() ),
               y = matches( selector_y() ),
               lb = matches( selector_lb() ),
               ub = matches( selector_ub() ))
    } else {
      clean_cos_data %>%
```

```r
      select(matches( selector_x() ),
             matches( selector_y() )
             ) %>%
      select(x = matches( selector_x() ),
             y = matches( selector_y() ))
  }
})
# (c) create xts-formated table for use in dygraphs:
xts_selected_data <- reactive ({
  xts(selected_data(),
      order.by = clean_cos_data[[ opts[["name_COSposix"]] ]])
})
# (d) create plots:
base_graph <- reactive({
  if(plot_bounds) {
    dygraph( xts_selected_data() ) %>%
    dyAxis("y",
           label = opts[["data_unit"]]) %>%
    dySeries("x",
             label = name_o,
             color = opts[["color_o"]]) %>%
    dySeries("y",
             label = name_s,
             color = opts[["color_s"]]) %>%
    dySeries("lb",
             label = name_lb,
             color = "grey80") %>%
    dySeries("ub",
             label = name_ub,
             color = "grey80")
  } else {
    dygraph( xts_selected_data() ) %>%
    dyAxis("y",
           label = opts[["data_unit"]]) %>%
    dySeries("x",
             label = name_o,
             color = opts[["color_o"]]) %>%
    dySeries("y",
             label = "name_s",
             color = opts[["color_s"]])
  }
})
output$hydrographs <- renderDygraph({
  base_graph() %>%
    dyRangeSelector(height = 20, strokeColor = "") %>%
    dyCrosshair(direction = "vertical") %>%
    dyOptions(includeZero = TRUE,
              retainDateWindow = TRUE,
              animatedZooms = TRUE)
})
# (e) get dygraph date bounds (switches):
selcted_from <- reactive({
  if (!is.null(input$hydrographs_date_window)) {
```

```
      input$hydrographs_date_window[[1]]
    }
  })
  selcted_to <- reactive({
    if (!is.null(input$hydrographs_date_window)) {
      input$hydrographs_date_window[[2]]
    }

  })
  # (f) extract time_window for the stats header:
  output$selected_timewindow <- renderText({
    if (!is.null(input$hydrographs_date_window))
      paste(strftime(selcted_from(), format = "%d %b %Y"),
            "-",
            strftime(selcted_to(), format = "%d %b %Y"),
            sep = " ")
  })
  # (g) calculate stats:
  sub_slctd <- reactive({
    if (!is.null(input$hydrographs_date_window))
      xts_selected_data()[paste(strftime(selcted_from(), format = "%Y-%m-%d-%H-%M"),
                                strftime(selcted_to(), format = "%Y-%m-%d-%H-%M"),
                                sep = "/")]
  })
  out_of <- reactive({
    if (!is.null(input$hydrographs_date_window)) {
        map_df(of_metrics, function(of_,x,y) of_(x,y),
               x = sub_slctd()$x,
               y = sub_slctd()$y ) #serve_of( sub_slctd()$x,sub_slctd()$y )
    }
  })

  output$slctd_OF <- renderTable(out_of())
  # (h) exit when user clicks on done
   # When the Done button is clicked, return a value
  observeEvent(input$done, {
    returnValue <- list(
      selected_time = c(strftime(selcted_from(), format = "%Y-%m-%d-%H-%M"),strftime(selcted_to(), fo
      selected_data = data.frame(date = index(sub_slctd()),
                                 coredata(sub_slctd())),
      selected_of = out_of()
    )
    stopApp(returnValue)
  })
}
```

The `miniUI` is quite spartan. There is an `miniButtonBlock` that allows to select different basin, as as the `dygraph` output (i.e `hydrographs`) for the interactive exploration of the o and s data. The formatted table (`slctd_OF`) displays the different objective functions, that can be given to `judge_explore` .

```
ui <- miniPage(
  miniButtonBlock(selectInput("basin_num",
                              "# basin:",
                              choices = d_nums,
```

```
                                selected = 1,
                                selectize = FALSE)),
    miniContentPanel(
      fillCol(
        flex = c(4,1),
        dygraphOutput("hydrographs", width = "100%", height = "100%"),
        fillCol(
          align = "center",
          textOutput("selected_timewindow"),
          tableOutput("slctd_OF")
        )
      )
    ),
    gadgetTitleBar("test")
  )
```

```
dyCrosshair <- function(dygraph,
                        direction = c("both", "horizontal", "vertical")) {
  dyPlugin(
    dygraph = dygraph,
    name = "Crosshair",
    path = system.file("plugins/crosshair.js",
                       package = "visCOS"),
    options = list(direction = match.arg(direction))
  )
}
```

```
  runGadget(ui,server)
}
```

6 Comparisons: Exploring Objective Functions II

This function is an expansion of the `judge_explore` function and assumes some familiarity with the latter. The idea is to explore two `cos_data` objects simultanously in the same way is in the `judge_explore` counterpart. Even if only one `data.frame` is handed over to the function, it copied and two `dygraphs` are shown below each other.

6.1 Code

```
# ---------------------------------------------------------------------------
# Code for judge_explore
# authors: Daniel Klotz, Johannes Wesemann, Mathew Herrnegger
# !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
```

```
  # ---------------------------------------------------------------------------
  #' Explore with Objective Functions
  #'
  #' Runs a Shiny Gadget which can be used to interactively explore two
  #' `cos_data` objects in terms of their respective objective functions.
  #'
  #' @import shiny
  #' @import miniUI
```

```r
  #' @import xts
  #' @import dplyr
  #' @import magrittr
  #' @import dygraphs
  #' @import pasta
  #' @importFrom purrr map_df
  #'
  #' @export

judge_explore2 <- function(d1,
                           d2 = NULL,
                           of_metrics = list(nse   = coscos::of_nse,
                                             kge   = coscos::of_kge,
                                             pbias = coscos::of_pbias,
                                             corr  = coscos::of_cor),
                       opts = coscos::viscos_options()) {
  # (I) pre-sets: ==============================================================
  if(!is.list(of_list)) {
    of_list = list(of_list)
  }
  if (is.null(d2)) {
    d2 <- d1
  }
  if (is.null(names(of_list))){
    names(of_list) <- paste("of", 1:length(of_list), sep = "_")
  }
  clean_data1 <- d1 %>%
    coscos::cook_cosdata(.) %>%
    coscos::remove_leading_zeros(.)
  clean_data2 <- d2 %>%
    coscos::cook_cosdata(.) %>%
    coscos::remove_leading_zeros(.)
  # convenience variables: ====================================================
  name_o <- opts[["name_o"]]
  name_s <- opts[["name_s"]]
  color_o <- opts[["color_o"]]
  color_s <- opts[["color_s"]]
  ylab <- opts[["data_unit"]]
  #
  name_lb <- opts[["name_lb"]]
  name_ub <- opts[["name_ub"]]
  names_d1 <- names(clean_data1) %>% tolower(.)
  names_d2 <- names(clean_data2) %>% tolower(.)
  #
  idx_names_d1 <- grepl(name_o, names_d1, ignore.case = TRUE)
  d_nums <- names_d1 %>%
      .[idx_names_d1] %>%
      gsub("\\D","",.) %>%
      as.integer(.) %>%
      unique(.)
  idx_names_d2 <- grepl(name_o, names_d2, ignore.case = TRUE)
  d_nums <- names_d2 %>%
      .[idx_names_d2] %>%
```

```
    gsub("\\D","",.) %>%
    as.integer(.) %>%
    unique(.)
# check for potential bounds: ==============================================
check_for_bounds <- function(names_data) {
  number_lb <- grepl(name_lb,
                     names_data,
                     ignore.case = TRUE) %>%
    sum(.)
  number_ub <- grepl(name_ub,
                     names_data,
                     ignore.case = TRUE) %>%
    sum(.)
  #
  plot_bounds <- FALSE
  if( (number_lb > 0) & (number_ub > 0)) {
    number_obs <- grepl(name_o,
                        names_d1,
                        ignore.case = TRUE) %>%
      sum(.)
    number_sim <- grepl(name_s,
                        names_d1,
                        ignore.case = TRUE) %>%
      sum(.)
    if (number_lb != number_ub) {
      stop("number of available bounds is not the same!" %&&%
             "#lb=" %&% number_lb %&&%
             "#ub=" %&% number_ub)
    } else if ((number_lb != number_obs) | (number_lb != number_sim)) {
      stop("Number of bounds is not the same as the o/s data!" %&&%
            "#bounds=" %&% number_lb %&&%
             "#obs=" %&% number_obs %&&%
             "#sim=" %&% number_sim)
    } else {
      plot_bounds <- TRUE # switch: plot bounds
    }
  }
  return(plot_bounds)
}
plot_bounds_d1 <- check_for_bounds(names_d1)
plot_bounds_d2 <- check_for_bounds(names_d2)
# app functions: ==========================================================
# subselect data: ##########################################################
subselect <- function(cos_data,
                      basin_number,
                      plot_bounds) {
  unique_data_names <- names(cos_data) %>%
    tolower(.) %>%
    gsub("\\d", "", .) %>%
    unique(.)
  x_string <- unique_data_names[ grep(name_o, unique_data_names) ]
  y_string <- unique_data_names[ grep(name_s, unique_data_names) ]
  x_sel<-  x_string %&% basin_number %&% "$"
```

```r
  y_sel <- y_string %&% basin_number %&% "$"
  if(plot_bounds) {
    lb_string <-  unique_data_names[ grep(name_lb, unique_data_names) ]
    ub_string <-  unique_data_names[ grep(name_ub, unique_data_names) ]
    lb_sel <- lb_string %&% basin_number %&% "$"
    ub_sel <- ub_string %&% basin_number %&% "$"
    selected_data <- cos_data %>%
      select(x = matches( x_sel ),
             y = matches( y_sel),
             lb = matches( lb_sel ),
             ub = matches( ub_sel))
    } else {
      selected_data <- cos_data %>%
        select(x = matches( x_sel ), y = matches( y_sel ))
    }
  selected_data %<>%
    xts(.,order.by = cos_data[[ opts[["name_COSposix"]] ]])
}
# plotting: ################################################################
create_dygraph <- function(plot_data, plot_group,plot_bounds){
  if(plot_bounds) {
    base_graph <- dygraph(plot_data, group = plot_group) %>%
      dyAxis("y", label = ylab) %>%
      dySeries("x", label = name_o, color = color_o) %>%
      dySeries("y", label = name_s, color = color_s) %>%
      dySeries("lb", label = name_lb, color = "grey80") %>%
      dySeries("ub",label = name_ub,color = "grey80")
  } else {
    base_graph <- dygraph(plot_data, group = plot_group) %>%
      dyAxis("y", label = ylab) %>%
      dySeries("x", label = name_o, color = color_o) %>%
      dySeries("y", label = name_s, color = color_s)
  }
}
```

```r
# (V) Define App: ==========================================================
server <- function(input, output, session) {
  xts_selected_data1 <- reactive({
    subselect(clean_data1, input$basin_num, plot_bounds_d1)
    })
  xts_selected_data2 <- reactive({
    subselect(clean_data2, input$basin_num2, plot_bounds_d2)
    })

  # (d) create plots:
  upper_graph <- reactive({
    create_dygraph(xts_selected_data1(), "test", plot_bounds_d1)
    })
  lower_graph <- reactive({
    create_dygraph(xts_selected_data2(), "test", plot_bounds_d1)
    })
  output$hydrographs <- renderDygraph({
    upper_graph() %>%
      dyOptions(includeZero = TRUE,
```

```
                retainDateWindow = TRUE,
                animatedZooms = TRUE)
  })
  output$hydrographs2 <- renderDygraph({
    lower_graph() %>%
      dyRangeSelector(height = 20, strokeColor = "") %>%
      dyOptions(includeZero = TRUE,
                retainDateWindow = TRUE,
                animatedZooms = TRUE)
  })

  # (e) get dygraph date bounds (switches):
  selcted_from <- reactive({
    if (!is.null(input$hydrographs_date_window)) {
      input$hydrographs_date_window[[1]]
    }
  })
  selcted_to <- reactive({
    if (!is.null(input$hydrographs_date_window)) {
      input$hydrographs_date_window[[2]]
    }
  })

  # (f) extract time_window for the stats header:
  output$selected_timewindow <- renderText({
    if (!is.null(input$hydrographs_date_window))
      paste(strftime(selcted_from(), format = "%d %b %Y"),
            "-",
            strftime(selcted_to(), format = "%d %b %Y"),
            sep = " ")
  })

  # (g) calculate stats:
  sub_slc_cos1 <- reactive({
    if (!is.null(input$hydrographs_date_window))
      xts_selected_data1()[paste(strftime(selcted_from(), format = "%Y-%m-%d-%H-%M"),
                                 strftime(selcted_to(), format = "%Y-%m-%d-%H-%M"),
                                 sep = "/")] })
  sub_slc_cos2 <- reactive({
    if (!is.null(input$hydrographs_date_window))
      xts_selected_data2()[paste(strftime(selcted_from(), format = "%Y-%m-%d-%H-%M"),
                                 strftime(selcted_to(), format = "%Y-%m-%d-%H-%M"),
                                 sep = "/")] })
  out_of <- reactive({
    if (!is.null(input$hydrographs_date_window)) {
        cbind.data.frame(
        map_df(of_list, function(of_,x,y) of_(x,y),
               x = sub_slc_cos1()$x,
               y = sub_slc_cos1()$y ) %>%
          set_names("d1" %-% names(.)),
        "-" = "-",
        map_df(of_list, function(of_,x,y) of_(x,y),
               x = sub_slc_cos2()$x,
```

```
                y = sub_slc_cos2()$y ) %>%
            set_names("d2" %-% names(.))
          )
      } })

    output$slctd_OF <- renderTable(out_of())
    # (h) exit when user clicks on done:
    observeEvent(input$done, {
      returnValue <- list(
        selected_time = c(strftime(selcted_from(), format = "%Y-%m-%d-%H-%M"),
                          strftime(selcted_to(), format = "%Y-%m-%d-%H-%M")),
        selected_data = data.frame(date = index( sub_slc_cos1() ),
                                   d1 = coredata( sub_slc_cos1() ),
                                   d2 = coredata( sub_slc_cos2() )),
        selected_of = out_of()
      )
      stopApp(returnValue)
    })
  }
```

The `miniUI` is quite spartan. There is an `miniButtonBlock` that allows to select different basin, as as the `dygraph` output (i.e `hydrographs`) for the interactive exploration of the o and s data. The formatted table (`slctd_OF`) displays the different objective functions, that can be given to `explore_cos_data` .

```
  ui <- miniPage(
    miniButtonBlock(selectInput("basin_num",
                                "d1:",
                                choices = d_nums,
                                selected = 1,
                                selectize = FALSE,
                                width = '50%'),
                    selectInput("basin_num2",
                                "d2:",
                                choices = d_nums,
                                selected = 1,
                                selectize = FALSE,
                                width = '50%')),
    miniContentPanel(
      fillCol(
          flex = c(4,4,2),
          dygraphOutput("hydrographs", width = "100%", height = "100%"),
          dygraphOutput("hydrographs2", width = "100%", height = "100%"),
          fillCol(
            align = "center",
            tableOutput("slctd_OF"))
      )
    ),
    gadgetTitleBar("test")
  )
```

```
  runGadget(ui,server)
}
```